ARTIFICIAL INTELLIGENCE

Lab 04: Data Manipulation with Python Pandas

Hands-on lab: DataFrames, filtering, grouping, missing data, merging, and file I/O

Course	Introduction to Artificial Intelligence / Data Science
Lab No.	04
Topic	Data Manipulation using the Pandas Library
Tool Required	Python 3.x, pandas, Jupyter Notebook / any Python IDE
Duration	90–120 minutes

Lab Objectives

By the end of this lab, students will be able to:

- Construct pandas DataFrames from Python dictionaries and understand tabular data representation.
- Inspect a dataset's structure, size, and summary statistics using pandas' built-in exploration methods.
- Select, filter, and slice rows and columns using label-based and position-based indexing.
- Add, modify, and remove columns to transform raw data into analysis-ready form.
- Sort records and summarize grouped data using aggregation functions (sum, mean, max, count).
- Detect and handle missing values through removal or imputation strategies.
- Read from and write to CSV files as part of a basic data I/O workflow.
- Apply custom logic to a column using the .apply() method with lambda functions.
- Combine multiple datasets using merge and concatenate operations, mirroring real-world data integration tasks.

These skills form the foundation of the data preprocessing stage in any AI or machine learning pipeline — a step that typically consumes the majority of a data scientist's project time.

Part A — Environment Setup

Import the pandas library before running any of the examples or tasks below.

```
import pandas as pd
```

Part B — Core Pandas Concepts

Work through each concept below in order. Run every snippet yourself rather than only reading it — pandas is best learned by observing how each method changes the output.

1. Creating a DataFrame

A DataFrame is the primary pandas data structure — a labeled, two-dimensional table, conceptually similar to an Excel sheet or a SQL table.

```
data = {
    'Name': ['Ali', 'Sara', 'Ahmed'],
    'Age': [20, 21, 22],
    'Marks': [75, 88, 92]
}
df = pd.DataFrame(data)
print(df)
```

2. Inspecting the Table

Before any analysis, always inspect a new dataset — its shape, column names, data types, and summary statistics.

```
df.head()        # first 5 rows
df.head(2)       # first 2 rows
df.tail(2)       # last 2 rows
df.shape         # (rows, columns)
df.columns       # column names
df.info()        # types + missing
df.describe()    # statistics
```

3. Selecting Columns

A single column returns a Series; a list of columns returns a DataFrame.

```
df['Name']               # one column (Series)
df[['Name', 'Marks']]    # two columns (DataFrame)
```

4. Selecting Rows

Use .iloc[] for position-based selection and .loc[] for label-based selection.

```
df.iloc[0]        # first row by position
df.iloc[1]        # second row
df.iloc[0:2]      # first two rows
df.loc[1]         # row by label
```

5. Filtering Rows

Boolean conditions inside square brackets act as a filter — this is the pandas equivalent of a SQL WHERE clause.

```
df[df['Marks'] > 80]
df[df['Age'] == 21]
df[(df['Marks'] > 80) & (df['Age'] > 20)]
```

6. Adding and Removing Columns

New columns can be assigned directly, and existing ones removed with .drop().

```
df['City'] = ['Lahore', 'Karachi', 'Islamabad']    # add
df['Bonus'] = df['Marks'] * 0.1                     # add from other column
df = df.drop('Bonus', axis=1)                       # remove
```

7. Sorting

Sort a DataFrame by one or more columns using .sort_values().

```
df.sort_values('Marks')                        # small to big
df.sort_values('Marks', ascending=False)       # big to small
```

8. Grouping and Aggregation

groupby() splits data into groups, then an aggregation function (sum, mean, max, count) summarizes each group — a core pattern for reporting and analytics.

```
data = {
    'City': ['NYC', 'LA', 'NYC', 'LA'],
    'Sales': [100, 200, 150, 250]
}
df = pd.DataFrame(data)

df.groupby('City')['Sales'].sum()      # total
df.groupby('City')['Sales'].mean()     # average
df.groupby('City')['Sales'].max()      # max
df.groupby('City')['Sales'].count()    # count
```

9. Handling Missing Data

Real-world datasets are rarely complete. pandas represents missing values as NaN and provides tools to detect and handle them.

```
data = {
    'Name': ['A', 'B', 'C'],
    'Score': [50, None, 70]
```

```
}
df = pd.DataFrame(data)

df.isnull()                # True/False
df.isnull().sum()          # count missing
df.dropna()                # remove rows with missing
df['Score'].fillna(0)      # fill with 0
df['Score'].fillna(df['Score'].mean())   # fill with average
```

10. Saving and Loading Files

pandas reads and writes CSV files directly, making it the standard bridge between raw data files and Python analysis.

```
df.to_csv('data.csv', index=False)     # save
df = pd.read_csv('data.csv')           # load
print(df)
```

11. Modifying Values

Existing values can be updated in place using .loc[], vectorized arithmetic, or .rename().

```
df.loc[0, 'Marks'] = 100              # change one cell
df['Marks'] = df['Marks'] + 5         # add 5 to all marks
df.rename(columns={'Marks': 'Score'}, inplace=True)    # rename column
```

12. Applying Functions

.apply() runs a custom function (often a lambda) across every value in a column — useful for deriving new categorical columns.

```
df['Result'] = df['Marks'].apply(lambda x: 'Pass' if x >= 80 else 'Fail')
```

13. Counting Unique Values

value_counts() and unique() summarize the distribution of categorical columns.

```
df['City'].value_counts()      # count each unique value
df['City'].unique()            # unique values
```

14. Merging Tables

merge() combines two DataFrames on a shared key column, similar to a SQL JOIN.

```
df1 = pd.DataFrame({'ID': [1, 2, 3], 'Name': ['A', 'B', 'C']})
df2 = pd.DataFrame({'ID': [1, 2, 4], 'Score': [80, 90, 85]})

pd.merge(df1, df2, on='ID')                  # common rows
pd.merge(df1, df2, on='ID', how='left')      # keep all from df1
pd.merge(df1, df2, on='ID', how='outer')     # keep all from both
```

15. Concatenating Tables

concat() stacks DataFrames on top of each other — useful for combining batches of records with the same columns.

```
pd.concat([df1, df1], ignore_index=True)
```

Part C — Lab Tasks

Complete the following tasks in a single notebook or script. Attempt each task independently before checking Part B for reference — this is where the concepts are put into practice.

Task 1 — Create the Table

- Build a DataFrame with Name: Ali, Sara, Ahmed; Age: 20, 21, 22; Marks: 75, 88, 92.
- Print the table.

Task 2 — Inspect the Table

- Using Task 1's table, print the first 2 rows.
- Print the shape and the column names.

Task 3 — Select Columns

- Print only the Name column.
- Then print the Name and Marks columns together.

Task 4 — Select a Row

- Print the second row (index 1) using .iloc[].

Task 5 — Add a Column

- Add a column City with values ['Lahore', 'Karachi', 'Islamabad'].
- Print the updated table.

Task 6 — Filter Rows

- Print only the students with Marks greater than 80.

Task 7 — Sort the Table

- Sort the table by Marks from highest to lowest.

Task 8 — Group and Aggregate

- Build the Team/Points table shown in Part B, Section 8.
- Group by Team and print the total points per team.

Task 9 — Handle Missing Data

- Build the Name/Score table with one missing Score.
- Count the missing values, fill them with 0, and print the table.

Task 10 — Save and Reload a File

- Save any of your tables as mytask.csv.
- Load it back using pd.read_csv().
- Print the first 3 rows.

Task 11 — Derive a Result Column

- Add a Result column: 'Pass' if Marks ≥ 80, otherwise 'Fail'.

Task 12 — Average by Group

- Build the City/Sales table from Part B, Section 8 (5 rows).
- Print the average sales per city.

Task 13 — Merge Two Tables

- Merge df1 and df2 (from Part B, Section 14) on ID.
- Print the merged result.

Task 14 — Rename a Column

- Rename the Marks column to Score in Task 1's table.

Task 15 — Mini Project — Student Report Card

- Build a table of 5 students with columns: Name, Math, Science, English.
- Add Total = Math + Science + English.
- Add Average = Total / 3.
- Add Result = 'Pass' if Average ≥ 50 else 'Fail'.
- Sort the table by Total in descending order.
- Print the top 3 students.
- Print the count of Pass vs Fail using .value_counts().

Submission Checklist

- All 15 tasks attempted with visible output for each (screenshots or printed cells).
- Code is commented where logic is non-obvious.

- mytask.csv (from Task 10) submitted alongside the notebook/script.
- Mini Project (Task 15) includes all six required steps in order.